Lab 1

Introduction to HTML, CSS, Bootstrap & AI-Assisted Development

Introduction

Web development is the foundation of modern software systems. Every application—whether it is a simple website or a complex AI-powered platform, relies on a user interface that allows users to interact with the system. This lab will begin your journey into full-stack development by building a structured and styled web page using HTML, CSS, and Bootstrap.

HTML (HyperText Markup Language) provides the structure of a web page, defining elements such as headings, paragraphs, images, and links. CSS (Cascading Style Sheets) is used to control the appearance of these elements, including layout, colors, and spacing. Bootstrap is a widely used front-end framework that simplifies styling and responsive design using prebuilt classes.

In addition to foundational web development skills, this lab introduces how modern web interfaces are designed to interact with AI systems. Rather than using AI only as a coding assistant, students will begin thinking about how front-end applications serve as the interaction layer between users and intelligent systems.

In real-world applications, front-end interfaces are responsible for:

- collecting user input (text, images, actions)
- sending requests to AI-powered services
- displaying AI-generated responses in a structured and user-friendly way

This lab focuses on building the user interface layer, which will later connect to AI-driven features in future labs.

By the end of this lab, students will have created a personal profile web page, applied styling using Bootstrap, and built a foundational user interface designed to integrate with AI-driven functionality in later labs.

Lab Objectives

By completing this lab, students will be able to:

- Understand the basic structure of an HTML document
- Create and manage project files using Visual Studio Code
- Build a simple web page using HTML elements
- Apply basic styling using CSS and Bootstrap
- Use a web browser to test and inspect web pages
- Develop a front-end interface that can be extended to support AI-driven features in future labs
- Use AI-assisted development tools such as GitHub Copilot to improve coding efficiency, including generating code snippets, identifying errors, and suggesting improvements

Required Software

For this lab, students should install the following software:

- Visual Studio Code
- A modern web browser (Google Chrome recommended)
- (Optional) GitHub Copilot extension in VS Code

Installing Visual Studio Code

1. Go to: https://code.visualstudio.com/
2. Click Download for Windows (or Mac)
3. Run the installer and follow the default setup instructions

Opening VS Code

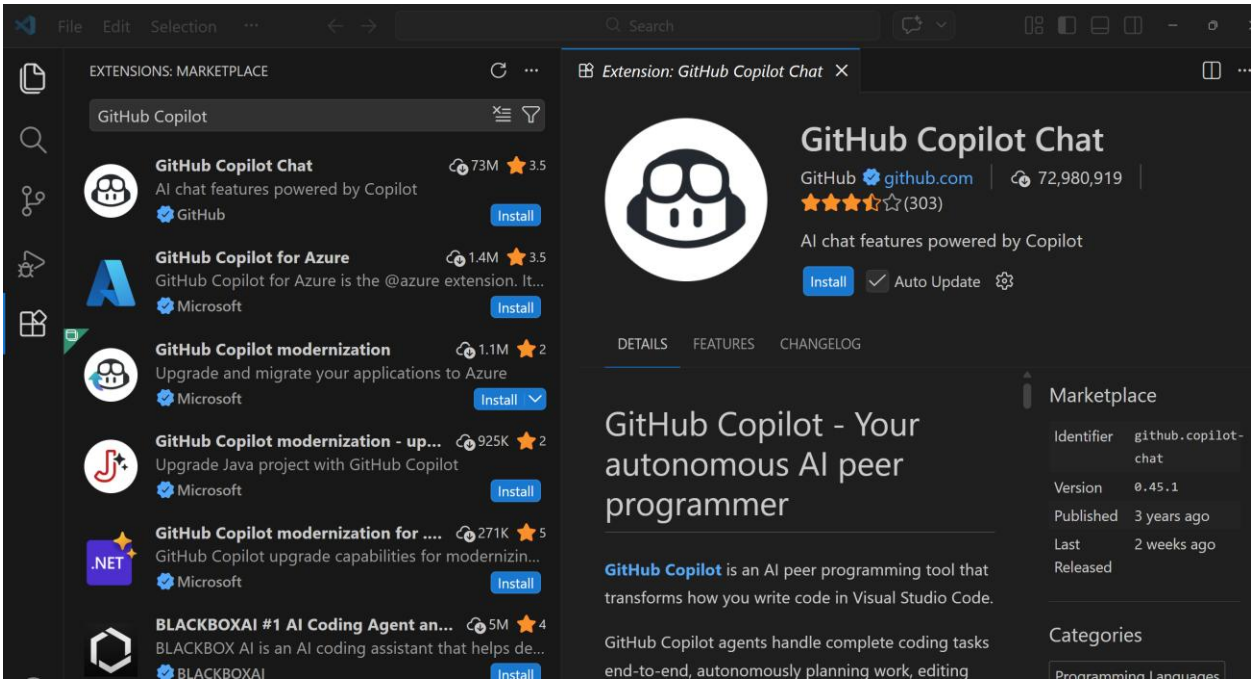
After installation:

1. Launch Visual Studio Code
2. You should see the main interface with:
    - Menu bar
    - Explorer panel (left side)
    - Editor area

(Optional) Installing GitHub Copilot

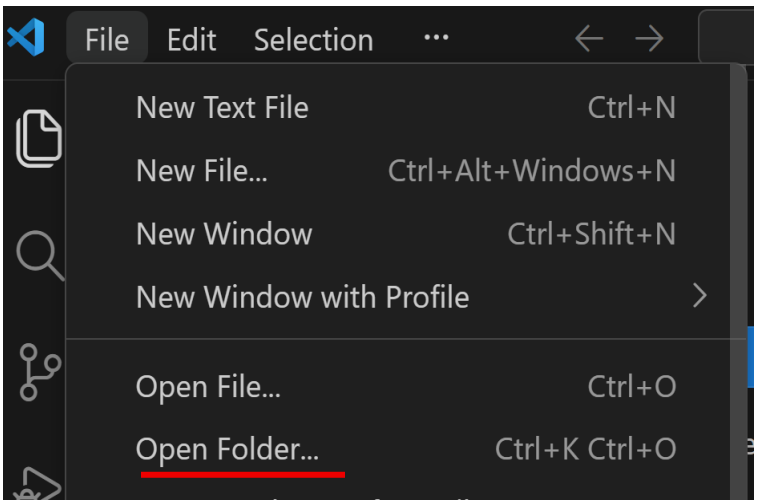
To enable AI-assisted development:

1. Open Extensions (left sidebar icon)
2. Search for GitHub Copilot Chat (GitHub Copilot is deprecated)
3. Click Install
4. Sign in with your GitHub account



Project Setup

1. Create Project Folder named: homework1
(You can also create and open a folder directly from VS code- Open Folder option)



2. Create Project Files

Inside the **homework1** folder, create:

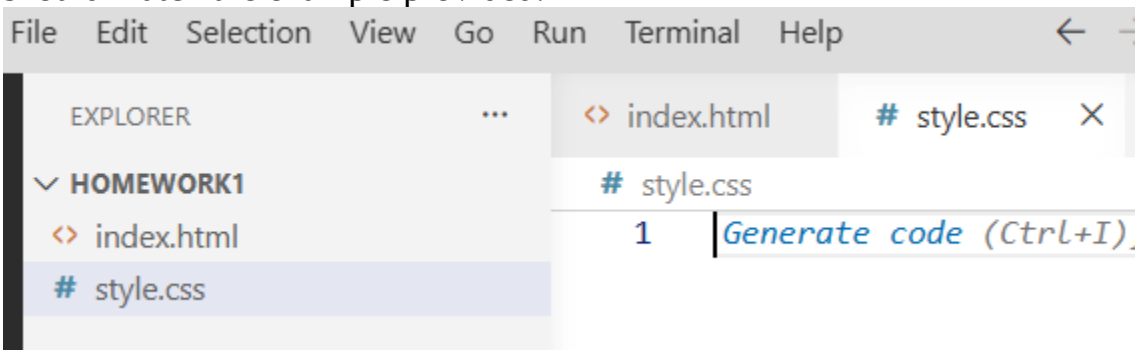
```
index.html
styles.css
```

3: Open Folder in VS Code

1. Click File → Open Folder
2. Select the **homework1** folder
3. Click Select Folder

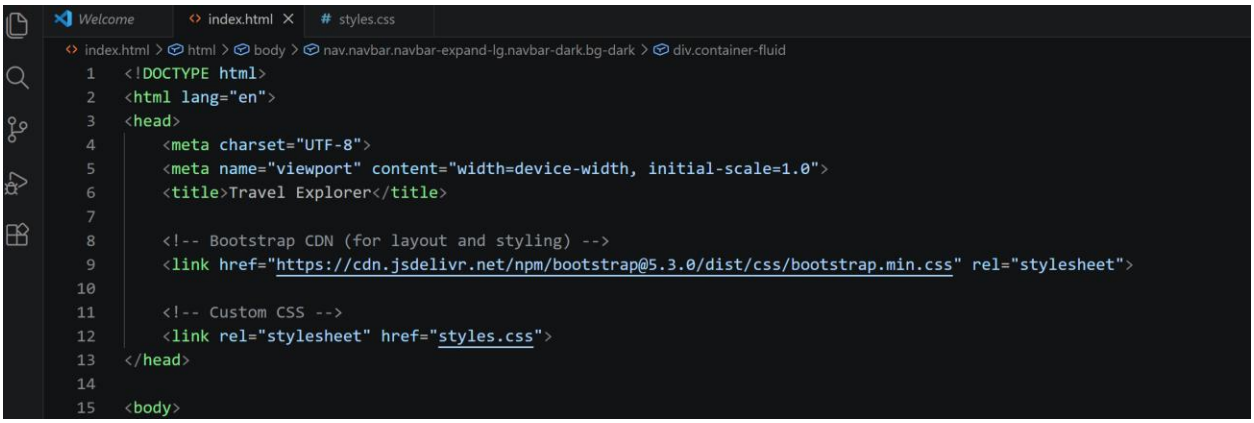
Working on Website Coding Files (HTML, CSS, Bootstrap) of Landing Page

Click on index.html to open it in the editor. Set up the basic HTML structure, then add your code to index.html as shown below. Your application's landing page should match the example provided.



Complete the HTML file first, then create and apply the styles.css file. You may use Bootstrap to handle most of the styling and layout, including components such as cards.

index.html




```
<!-- ================ NAVBAR ================ -->
<!-- NOTE: This is a reusable UI component used in most modern apps -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
        <a class="navbar-brand" href="#">🌎 Travel Explorer</a>

        <!-- Burger Button -->
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarContent">
            <span class="navbar-toggler-icon"></span>
        </button>

        <!-- Collapsible Menu -->
        <div class="collapse navbar-collapse" id="navbarContent">
            <ul class="navbar-nav ms-auto">
                <li class="nav-item"><a class="nav-link" href="#">Home</a></li>
                <li class="nav-item"><a class="nav-link" href="#">Destinations</a></li>
                <li class="nav-item"><a class="nav-link" href="#">Plan Trip</a></li>
            </ul>
        </div>
    </div>
</nav>
```

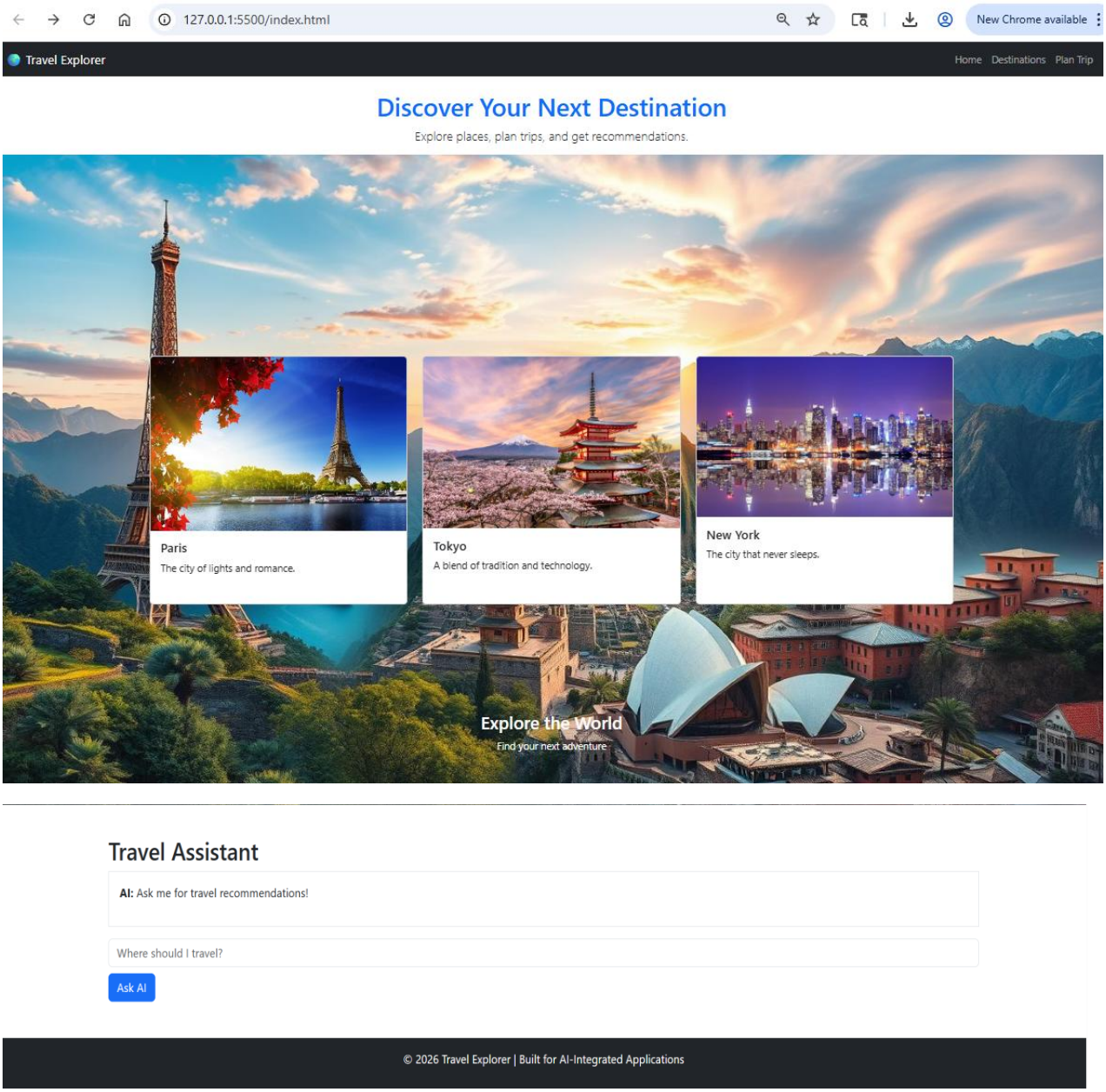
Complete the rest of the HTML/CSS/Bootstrap based on the example.

If GitHub Copilot is installed (more details will be shown in Using AI-Assisted Development section):

1. Start typing <html>
2. Observe suggested code completion
3. Press **Tab** to accept suggestion

Note: The Travel Assistant AI section mimics how AI systems respond.
Future labs will replace this with real API calls.

Example of homework1 Home Page



Multi-Page Applications

Most real-world web applications are not a single page. Instead, they are composed of multiple pages that are connected through navigation.

In this section, you will expand your project into a multi-page website.

1. Inside your **homework1** folder, create the following files:
   - destinations.html
   - plan.html
2. Copy the structure from index.html and paste into both new html files.
3. Update Navbar Links

```
<ul class="navbar-nav ms-auto">
    <li class="nav-item"><a class="nav-link" href="index.html">Home</a></li>
    <li class="nav-item"><a class="nav-link" href="destinations.html">Destinations</a></li>
    <li class="nav-item"><a class="nav-link" href="plan.html">Plan Trip</a></li>
</ul>
```

4. Add Content to Each Page

(Optional) Using AI-Assisted Development (GitHub Copilot)

Modern development environments often include AI-assisted tools that help developers by suggesting code, completing repetitive structures, and identifying common patterns.

In this lab, you may optionally practice using GitHub Copilot to support your development workflow.

Step 1: Generate HTML Structure with Copilot

1. Open a new file (e.g., plan.html)
2. Type:

   <!DOCTYPE html>

3. Press Enter
4. Observe suggested structure
5. Press Tab to accept

Step 2: Generate a Section Using Comments

6. Type the following comment:
   `<!-- create a Bootstrap form for travel planning -->`

7. Press Enter and observe suggestions.

Review and Modify AI Suggestions

Important: Do NOT blindly accept suggestions. Always review generated code, and modify as needed

Task: Review and refine AI-generated code before integrating it into your project.

If GitHub Copilot is used, students must provide evidence of both the original generated code and the final edited version.

Introduction to API-Based AI Systems

Modern AI applications are not standalone front-end systems. They rely on communication between:

1. Frontend (User Interface)
2. Backend Server
3. AI Model / API

Basic Workflow:

User Input → Frontend → API Request → AI System → Response → Frontend Display

Explanation: The user enters input (e.g., "Where should I travel?"). The frontend sends a request to a server, which then communicates with an AI service. The response is returned to the frontend and displayed in the user interface.

Real-World Example

Applications like ChatGPT follow this same structure.

Understanding API Calls (Preview)

Concept: What is an API?

An API (Application Programming Interface) allows different systems to communicate.
In future labs, your frontend will send requests to:

- backend servers
- AI services

Understanding API Calls (Preview)
An API (Application Programming Interface) allows different systems to communicate.
In future labs, your frontend will send requests to:

- backend servers
- AI services

Example of API Call (Review Only)

```
fetch("https://api.example.com/ai", {
    method: "POST",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify({
        message: "Recommend a travel destination"
    })
})
.then(response => response.json())
.then(data => console.log(data));
```

Explanation
fetch() sends a request to a server. The POST method is used to send data, and the body contains the user's input. The server processes the request and returns a response, which may include AI-generated data.

Note: You are NOT required to implement APIs in this lab. This is a preview of future development. Your current goal is to:

- build the UI
- understand the data flow

Before moving forward, ensure:

- Your chat interface works
- Code is structured into functions
- You understand the API workflow concept

Preparing the Frontend for AI Integration (Concept Only)

Why Prepare for AI Now?

Modern applications are increasingly powered by AI, but AI is not directly embedded in HTML pages. Instead, it works through external services (APIs).

In this lab, you will prepare your frontend so that it can connect to AI systems in future labs

How AI Will Work in Future Labs.

User Input → Frontend → API Request → AI Service → Response → Display

Implementing a Mock AI Interaction:

Add AI Interaction Section on HTML file

```
    <div class="container mt-5">
    <h2>Travel Assistant</h2>

    <div id="chat-box" class="border p-3 mb-3">
        <p><strong>AI:</strong> Ask me for travel suggestions!</p>
    </div>

    <input id="user-input" type="text" class="form-control" placeholder="Where should I travel?">

    <button class="btn btn-primary mt-2" onclick="sendMessage()">Ask</button>
</div>
```

Add Mock JavaScript Logic

```
/*
IMPORTANT:
This is a MOCK implementation.

There is NO real AI connection yet.
We are simulating how an AI system behaves.
*/

function sendMessage() {
    let input = document.getElementById("user-input").value;
    let chatBox = document.getElementById("chat-box");

    // Display user input
    chatBox.innerHTML += "<p><strong>You:</strong> " + input + "</p>";

    // Simulated AI response
    let response = getMockResponse(input);

    chatBox.innerHTML += "<p><strong>AI:</strong> " + response + "</p>";

    document.getElementById("user-input").value = "";
}

/*
This function simulates AI behavior.
In future labs, this will be replaced with an API call.
*/
function getMockResponse(input) {

    if (input.toLowerCase().includes("beach")) {
        return "You might enjoy visiting Hawaii!";
    }

    if (input.toLowerCase().includes("city")) {
        return "Consider exploring New York or Tokyo!";
    }

    return "I recommend exploring popular destinations!";
}
```

References

Full Stack Web Development Bootcamp. (2026). *Course materials and instructional content*.

ChatGPT reference:

ChatGPT. (2026). AI-assisted web development and frontend concepts.